Step-by-Step Clean Deployment Script

Here are the complete commands. They use explicit variables to prevent any truncation issues on your screen:

1. Download the Installer

This command forces curl to follow the GitHub release redirects (-L) and saves the exact Apple Silicon binary package locally: [1]

bash

```
curl -L -o Miniforge3-MacOSX-arm64.sh "https://github.com"
```

Use code with caution.

2. Run the Installer

Execute the script to start the local setup wizard: [1]

bash

```
sh Miniforge3-MacOSX-arm64.sh
```

Use code with caution.

- Press Space to scroll through the license terms.
- When prompted to choose an installation path, press Enter to accept the default user home directory.
- When asked *"Do you wish the installer to initialize Miniforge3 by running conda init?"*, type **yes**. [1, 2, 3]

3. Reload Your Terminal

Close your terminal entirely and open a brand-new window to activate the conda environmental variables. You will see a (base) tag at the beginning of your terminal command prompt.

4. Build Your LeRobot Environment

Navigate to your existing repository folder and create the stable sandbox environment running the optimal Python 3.10 framework:

bash

```
cd path/to/lerobot
conda create -n lerobot_conda python=3.10 -y
conda activate lerobot_conda
```

Use code with caution.

5. Execute Core Package Installations

Because this environment runs Python 3.10, pip pulls pre-compiled binary wheels immediately, completely bypassing any Bazel source compilation workflows:

bash

```
pip install -e ".[aloha, mujoco]"
```

Use code with caution.

Let me know if pip install successfully finishes using this explicit GitHub release path so we can immediately test your environment by streaming a virtual simulation task!

AI responses may include mistakes. Learn more

-
-

GitHub

conda-forge/miniforge - GitHub
For unattended installs, please use the SH installers mentioned in the section above. As part of a CI pipeline If you wish to down...



GitHub

robotology-superbuild/doc/install-miniforge.md at master
all, download the installer and install it in its default location: To use the conda command, you need to add the ~/miniforge3/con...

Context7

Miniforge (conda-forge/miniforge)
Miniforge is a minimal installer for Conda and Mamba. The default installation path for Miniforge is within the user's home direct...

- 

-

Show all

I have downloaded the Miniforge3.sh now (its in the same lerobot local folder). When I execute it via bash, where will it install the conda env? Can I have it installed in the same lerobot folder (just like I did for the lerobot_env case?)

Yes, you can absolutely install it directly inside your lerobot folder!
By default, the Miniforge installer tries to install itself globally in your home directory (~/miniforge3). However, **you can change the path during the installation setup** to point exactly inside your current folder, mirroring your original venv behavior.

Here is how to run it step-by-step to keep everything self-contained:

Step 1: Run the Installer

Execute the shell script you just downloaded:

bash

```
sh Miniforge3-MacOSX-arm64.sh
```

Use code with caution.

Step 2: Set the Custom Local Path

1. Press Space to scroll through the license terms, and type yes to accept them.
2. The installer will then stop and present this prompt:text

   Miniforge3 will now be installed into this location:
3. /Users/yourname/miniforge3
4.
5. - Press ENTER to confirm the location
6. - Press CTRL-C to abort the installation
7. - Or specify a different location below
8.

   Use code with caution.

9. Do **not** press Enter. Instead, type your exact lerobot directory path followed by a new folder name (e.g., .conda_base) and press Enter:text

   [/Users/yourname/miniforge3] >>> /path/to/your/lerobot/.conda_base
10.

    Use code with caution.

Step 3: Initialize the Installer

At the very end of the installation, it will ask:

text

```
Do you wish the installer to initialize Miniforge3 by running
conda init? [yes|no]
```

Use code with caution.

Type **yes**. This modifies your terminal configuration file so your Mac knows where to find your new local Conda setup.

Step 4: Finalize and Verify

1. **Restart your terminal:** Close your current Terminal window completely and open a fresh one.
2. **Navigate back:** cd /path/to/your/lerobot
3. You should now see the (base) tag active on your command line!

Let me know once you have restarted your terminal and see the **(base)** tag. From there, I will show you how to create the specific **lerobot_conda** sub-environment directly inside your directory as well!

AI responses may include mistakes. Learn more

actually I might need to use miniforge (conda env) in multiple places yes? it suggests that I install it in /Users/anupamkaul/miniforge3. Is that OK? Can I install it here and then do the same pip equivalent thing where it "pulls" the main binaries from here to my container inside lerobot folder? Would that be a better approach?

Yes, installing it in /Users/anupamkaul/miniforge3 is the absolute best approach. You should definitely follow the installer's suggestion.
This is the standard, industry-preferred way to use Conda. It installs the "core engine" once in your home folder. From there, you can spin up dozens of isolated environments for different projects across your Mac.

The coolest part? Conda does something even better than copying binary files like venv does: it uses **hardlinks**. When you create an environment inside your lerobot folder, Conda links the binaries from your main /Users/anupamkaul/miniforge3 folder. It takes up **zero extra disk space** and is incredibly fast.
Here is the cleanest way to do exactly what you want, keeping the actual environment data safely tucked inside your lerobot folder just like venv did:

Step 1: Finish the Global Installation

1. Run the script again and press **ENTER** to accept the default /Users/anupamkaul/miniforge3 path.
2. When it asks to initialize by running conda init, type **yes**.
3. **Restart your terminal** (close it completely and open a new window). You should see the (base) tag.

Step 2: Create the Environment Directly Inside the LeRobot Folder

Instead of giving the environment a generic name, we will use the --prefix (or -p)

flag. This tells Conda to put the entire environment container directly inside a folder named .conda_env right inside your lerobot directory.
Navigate to your folder and run this:

bash

```
cd /path/to/your/lerobot

# Create the sandbox folder right here using Python 3.10
conda create --prefix ./.conda_env python=3.10 -y
```

Use code with caution.

Step 3: Activate Your Local Folder Environment

Because the environment is a local folder path instead of a global name, you activate it by pointing to that folder:

bash

```
conda activate ./.conda_env
```

Use code with caution.

(Your terminal prompt will change to show the full directory path, confirming you are safely locked into your local folder environment).

Step 4: Run the LeRobot Install Block

Now, run your installation. pip will pull the pre-compiled Python 3.10 Mac binaries from your global Miniforge engine and build everything seamlessly inside your local folder:

bash

```
pip install -e ".[aloha, mujoco]"
```

Use code with caution.

Let me know if the pip install finishes cleanly without throwing any Bazel errors using this localized prefix workflow! Once it wraps up, we will run the verification code to wake up the virtual humanoid simulator.

AI responses may include mistakes. Learn more

I followed your instructions and landed with this. Also, do we not need to install everything from scratch that lerobot needs to function? (something like the equivalent of a pip install -e ./. Anyways here is the error "pip install -e ".[aloha, mujoco]" Obtaining file:///Users/anupamkaul/other_git/hugging-face/lerobot Installing build dependencies ... done Checking if build backend supports build_editable ... done Getting requirements to build editable ... done Preparing editable metadata (pyproject.toml) ... done INFO: pip is looking at multiple versions of lerobot to determine which version is compatible with other requirements. This could take a while. ERROR: Package 'lerobot' requires a different Python: 3.10.22 not in '>=3.12' "

You ran directly into LeRobot's recent structural updates. The library **explicitly changed its minimum requirement to Python 3.12** (requires-python = ">=3.12"). [1]

Because we manually dropped back to Python 3.10 to fix the legacy labmaze error, pip blocked the installation entirely.
To answer your first question: **Yes, pip install -e ".[aloha, mujoco]" handles everything from scratch.** The -e . (editable mode) acts as the root installer, while the [aloha, mujoco] section pulls down all auxiliary simulation packages.
We need to align the configurations. Since LeRobot natively requires Python 3.12, we can build a **Python 3.12 environment** and resolve the problematic dependencies seamlessly.

Step 1: Wipe the 3.10 Environment

Make sure you are in your lerobot folder and delete the old environment completely:

bash

```
conda deactivate
rm -rf .conda_env
```

Use code with caution.

Step 2: Create a Pure Python 3.12 Environment Local Container

Create a fresh sandbox right inside your repository using **Python 3.12**, which perfectly satisfies LeRobot's core rules: [1]

bash

```
conda create --prefix ./.conda_env python=3.12 -y
conda activate ./.conda_env
```

Use code with caution.

Step 3: Install ffmpeg via Conda-Forge

LeRobot relies heavily on video token processing for visualization and dataset playback. Conda allows for a direct installation of this media layer to bypass native macOS compilation limits: [1, 2]

bash

```
conda install ffmpeg -c conda-forge -y
```

Use code with caution.

Step 4: Run the Complete Installation Block

Now that your local path matches Python 3.12, execute the core script. Modern Python 3.12 package wheels for dm-tree are readily accessible, which will allow you to clear the initial hurdle seamlessly: [1, 2]

bash

```
pip install -e ".[aloha, mujoco]"
```

Use code with caution.

🚨 What if labmaze still throws the Bazel error under 3.12?

If labmaze acts up during this clean step, turn off Bazel's bzlmod configuration to run the deployment script smoothly:

bash

```
BAZEL_LINKOPTS="--noenable_bzlmod" pip install --no-cache-dir
-e ".[aloha, mujoco]"
```

Use code with caution.

Let me know if your terminal wraps up without any conflicts under **Python 3.12**! Once it clears, we can run our verification script to load up the 3D physics pipeline.

AI responses may include mistakes. Learn more

-
-

  GitHub

  lerobot/pyproject.toml at main · huggingface/lerobot · GitHub
  PyTorch real-world robotics and machine learning requiring Python >=3.12. motor/robot/policy packages including feetech, diffusion...

-
-

  Hugging Face

  Installation
  Create a virtual environment with Python 3.12: Then activate your virtual environment, This usually installs ffmpeg 7.

- 

-
-
-
-

  GitHub

  Releases · huggingface/lerobot
  Aug 3, 2026 — v0.5.0 (Python 3.12+ required, Unitree G1 WBC implementation, cudnn_deterministic option), v3, streaming dataset).

- 
-

Show all

it built everything with no errors ! what next ?

Awesome! That is a huge milestone. Getting a clean build without errors on Apple Silicon means your local environment is officially configured and ready for deep learning robotics.
Now, let's step directly into **Validating, Visualizing, and Running simulations** on your M5 MacBook Pro. We will start with Imitation Learning **(IL)** since you already have LeRobot fully configured for it.

Step 1: Verify Hardware Acceleration

Before running heavy training scripts, let's ensure your installation can talk directly to your M5 Pro GPU (MPS backend) and that the MuJoCo engine wakes up.
Run this command in your terminal:

bash

```
python -c "import mujoco; print('🎉 MuJuCo Engine: READY'); import torch; print('🍏 Apple Silicon GPU Acceleration (MPS):', 'ACTIVE' if torch.backends.mps.is_available() else 'INACTIVE')"
```

Use code with caution.

*Make sure it prints **READY** and **ACTIVE** before moving forward.*

Step 2: Stream a Humanoid/Bimanual Dataset (No Robot Needed)

Before training an AI to mimic a human, you need data. Hugging Face hosts massive libraries of pre-recorded robot data. Let's download a standard bimanual

manipulation task (the arms/torso assembly of a humanoid robot) and visualize it locally.
Run the native LeRobot visualization script to play back a human demonstration episode right on your screen:

bash

```
python lerobot/scripts/visualize_dataset.py \
  --repo-id lerobot/aloha_sim_transfer_cube_human \
  --episode-index 0
```

Use code with caution.

- **What you should see:** A window will pop up showing a simulated 3D robot picking up a red cube and transferring it to its other hand. This is the exact dataset your AI policy will learn to replicate.

Step 3: Train Your First Policy via Imitation Learning

Now, let's train an ACT **(Action Chunking with Transformers)** policy—a state-of-the-art model used by modern humanoid manufacturers. Your M5 Pro chip will process the 3D physics states and camera pixels to train a neural network.
Run this training pipeline directly inside your terminal:

bash

```
python lerobot/scripts/train.py \
  policy=act \
  env=aloha \
  env.task=AlohaTransferCube-v0 \
  dataset_repo_id=lerobot/aloha_sim_transfer_cube_human \
  device=mps \
  training.batch_size=8 \
  training.offline_steps=500 \
  eval.n_episodes=2
```

Use code with caution.

- **What this does:** It tells LeRobot to use the act policy architecture, loads the virtual aloha environment, hooks into your Apple Silicon GPU (device=mps), and trains for a quick 500 steps while periodically running the virtual robot to evaluate how well it can move the cube.
- *Note:* We are using 500 steps just to test that the pipeline works without waiting hours. For a fully optimized agent, you would normally run 10,000+ steps.

Let me know if the **visualizer window opens successfully** or if the **training script begins iterating through steps**! Once we confirm Imitation Learning is moving smoothly, would you like to transition into installing **Genesis** to set up our Reinforcement Learning (RL) loop next?